

Mocking Dependencies in Spring Tests Using Mockito

Exercise 1: Mocking a Service Dependency in a Controller Test

Task: Test a Spring controller that fetches user data while the service dependency is replaced with a Mockito mock.

Explanation: The controller is tested independently. Mockito creates a fake UserService, predefined user data is returned, and the test verifies both the response and the service interaction.

User.java

```
package com.cognizant.springtesting;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {

    @Id
    private Long id;
    private String name;

    public User() {
    }

    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

UserService.java

```
package com.cognizant.springtesting;

import org.springframework.stereotype.Service;

@Service
public class UserService {
```

```

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}

```

UserController.java

```

package com.cognizant.springtesting;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

UserControllerTest.java

```

package com.cognizant.springtesting;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

public class UserControllerTest {

    @Test
    public void testGetUser() {

        UserService mockService = mock(UserService.class);
        User user = new User(1L, "Rekha");

        when(mockService.getUserById(1L)).thenReturn(user);

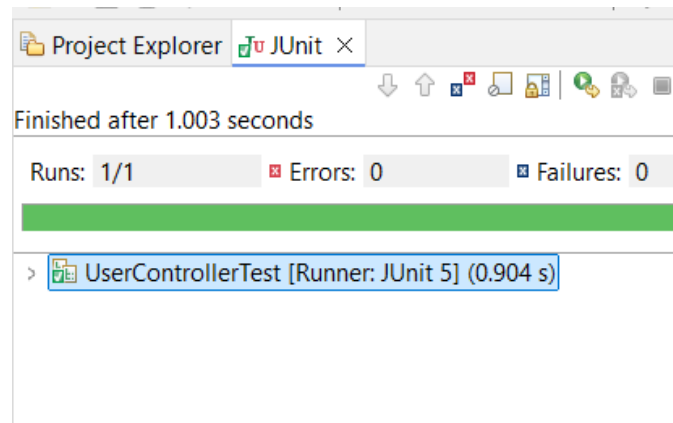
        UserController controller = new UserController(mockService);
        User result = controller.getUser(1L).getBody();

        assertEquals(1L, result.getId());
        assertEquals("Rekha", result.getName());
        verify(mockService).getUserById(1L);
    }
}

```

```
}  
}
```

Output



Exercise 2: Mocking a Repository in a Service Test

Task: Test a Spring service that fetches user data while the repository dependency is mocked with Mockito.

Explanation: The repository is replaced by a mock, so no real database is required. The test stubs `findById()`, calls the service method, checks the returned user, and verifies that the repository method was called.

User.java

```
package com.cognizant.springtesting;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {

    @Id
    private Long id;
    private String name;

    public User() {
    }

    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

UserRepository.java

```
package com.cognizant.springtesting;

import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
}
```

UserService.java

```
package com.cognizant.springtesting;

import org.springframework.stereotype.Service;

@Service
```

```

public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}

```

MockUserServiceTest.java

```

package com.cognizant.springtesting;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

import java.util.Optional;
import org.junit.jupiter.api.Test;

public class MockUserServiceTest {

    @Test
    public void testGetUserWithMockRepository() {

        UserRepository mockRepository = mock(UserRepository.class);
        User mockUser = new User(1L, "Rekha");

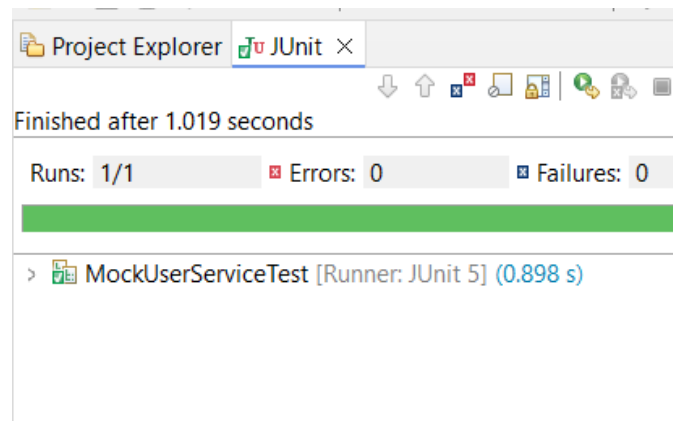
        when(mockRepository.findById(1L))
            .thenReturn(Optional.of(mockUser));

        UserService userService = new UserService(mockRepository);
        User result = userService.getUserById(1L);

        assertEquals(1L, result.getId());
        assertEquals("Rekha", result.getName());
        verify(mockRepository).findById(1L);
    }
}

```

Output



Exercise 3: Mocking a Service Dependency in an Integration Test

Task: Use Spring Boot integration-test support to load the application context, mock the service dependency, and test the controller endpoint with MockMvc.

Explanation: `@SpringBootTest` loads the Spring Boot application context. `@AutoConfigureMockMvc` provides MockMvc for HTTP-style testing, and `@MockBean` replaces the real `UserService` bean with a Mockito mock. This tests the controller within the Spring environment without using the real service or database.

User.java

```
package com.cognizant.springtesting;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {

    @Id
    private Long id;
    private String name;

    public User() {
    }

    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

UserService.java

```
package com.cognizant.springtesting;

import org.springframework.stereotype.Service;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

```

    }

    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}

```

UserController.java

```

package com.cognizant.springtesting;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

UserIntegrationTest.java

```

package com.cognizant.springtesting;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
public class UserIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    public void testGetUserWithMockService() throws Exception {

        User mockUser = new User(1L, "Rekha");
    }
}

```

```
when(userService.getUserById(1L))
    .thenReturn(mockUser);

mockMvc.perform(get("/users/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.id").value(1))
    .andExpect(jsonPath("$.name").value("Rekha"));
}
```

Output

